

SIG742: Modern Data Science

Extension Request

Students with difficulty in meeting the deadline because of proper reasons such as illness etc., must apply for an assignment extension with Great Learning one day prior to assessment deadline (Saturday). In such a case, an extension upto 1 week maybe granted at the discretion of the Academic Team. However, no further extension will be granted beyond this extended deadline.

Academic Integrity All assignment will be checked for plagiarism, and any academic misconduct will be reported to unit chair and university.

Generative AI Deakin's Policy and advices on responsible usage of Generative AI in your studies: <https://www.deakin.edu.au/students/study-support/study-resources/artificial-intelligence>

Instructions

Assignment Questions

There are total **2** parts in the assessment task:

Part 1 The first part will focus on the basic Python programming skills which includes the **data types**, the **control flow**, the **function**, the **modules** and **library** from **M02**.

Part 2 The second part is for those who are aiming to achieve 'High Distinction' (HD) for this assessment task, and it will focus on more advanced Python programming skills for data science on particular scenario. This part will require the knowledge covered in **M02** and also some part of the **M03**, in particular, **numpy** and beyond.

What to Submit?

You are required to submit the following completed files to the corresponding Assignment on Olympus:

`SIT742Task1.ipynb` The completed notebook with all the run-able code on all requirements.

In general, you need to complete, **save** the results of running, and submit your **notebook** from Python platform such as **Google Colab**. You need to clearly list the answer for each question, and the expected format from your notebook will be like in Figure 1.

```
Student ID: XXXXXXXXXXXXXXXX
Student Name: XXXXXXXXXXXXXXXXXXXX
Attempted High Distinction Question: Yes/No

Part 1

Answer 1.1

[ ] #YOUR CODE FOR QUESTION 1.1 IN PART 1 -- YOU ALSO NEED TO SAVE THE RESULTS OF RUN AS WELL UNDER THIS CELL

Answer 1.2

[ ] #YOUR CODE FOR QUESTION 1.2 IN PART 1 -- YOU ALSO NEED TO SAVE THE RESULTS OF RUN AS WELL UNDER THIS CELL

.....

Part 2

Answer 2

[ ] #YOUR CODE FOR QUESTION IN PART 2 -- YOU ALSO NEED TO SAVE THE RESULTS OF RUN AS WELL UNDER THIS CELL
```

Figure 1: Notebook Format

SIT742Task1Part2.avi (Only if you choose to work on *Part 2* of this assessment task) A video demonstration between 5 and 10 minutes, and the file format can be other common ones, such as ‘MKV’, ‘WMV’, ‘MOV’ etc.

If you are aiming to achieve ‘High Distinction’ (HD) and choose to work on *Part 2* of this assessment task, one important submission is a **short video** in which *You* orally present the solutions that you provide in the notebook and illustrate the running of code line by line.

SIT742Task1Part2report.pdf (Only if you choose to work on *Part 2* of this assessment task) A short essay report with no more than 500 words, and the file format can be other common ones, such as ‘.pdf’, ‘.doc’, ‘docx’ etc.

If you are aiming to achieve ‘High Distinction’ (HD) and choose to work on *Part 2* of this assessment task, another important submission is a **short essay** in which *You* will explain the code you wrote for the solution, any of the resource or material which has been helpful to resolve the problem, and illustrate if there is any alternative way to solve the problem. In addition, you will also need to follow the common essay writing structure which could be found in <https://www.deakin.edu.au/students/study-support/study-resources/academic-skills/essay-writing>

Part I

Python Programming

There are **8** questions in this part for total **80** marks, and each question is for **10** marks.

You are required to use **Google Colab** to finish all the coding in the *code block cell*, and **provide sufficient coding comments**, and also **save the result of running as well**.

Question 1.1

Write a method “is_divisible” that takes two integers, **m** and **n**. The method returns **True** if **m** is divisible by **n**, and returns **False** otherwise. Give three test cases for this function and three test cases should include cases that return values on **True** and **False**. Be sure to test your function well, including at least three test cases.

Question 1.2

Imagine that Python does not have the `!=` operator built in. Write a method “`not_equal`” that takes two variables(any variable type) and gives the same result as the `!=` operator. Obviously, you cannot use `!=` within your function! Test if your code works by thinking of examples and making sure the output is the same for your new method as `!=` gives you. Be sure to test your function well, including at least 3 test cases.

Question 1.3

List Comprehensions, you will need to perform and understand list comprehensions programming in below two small tasks.

- Write a list comprehension that prints out the possible results of three coin flips (how many results should be there?)
- Run this list comprehension in your notebook: `[x+y for x in [10,20,30] for y in [1,2,3]]`. Figure out what is going on here, and write a nested for loop that gives you the same result.

Question 1.4

Write a function “`find_digits`” that takes in a string and uses a list comprehension to return all the int digits in the string if any (assuming string only contain digits from 0 to 9). Be sure to test your function well, including at least 3 test cases.

Question 1.5

Write a list comprehension which solves the equation $y = 2x^2 - 1$ like Figure 2. Your solution should print out a list of $[x, y]$ pairs; use the domain $x \in [-3, 3]$ and the range $y \in [0, 30]$. x and y are all integers and you could use `range` for simulation on the x and y .

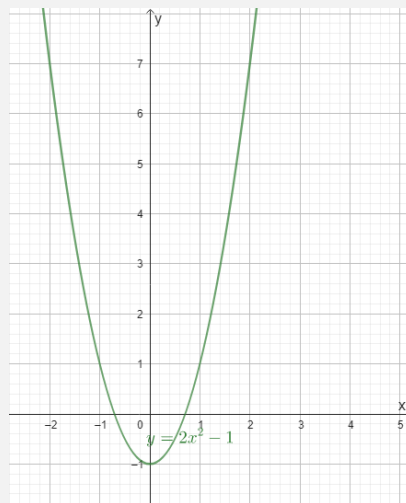


Figure 2: Equation of $y = 2x^2 - 1$

Question 1.6

Collision Detection of Balls:

For calculating collision, we only care about a ball’s position in space, as well as its size. We can store a ball’s position with the (x, y) coordinates of its center point, and we can calculate its size if we know its radius r . Thus, we represent a ball in 2D space as a tuple of (x, y, r) . To figure out if two balls are colliding, we need to compute the distance between their centers, then see if this distance is less than or equal to the sum of their radius. If so, they are colliding. Please define a function with

two list as inputs (two balls' position and radius [x,y,r]), test your function with three different cases.

Question 1.7

Scoring the words A set of score is given to you as below according to each English letters: LETTER_VALUES = 'a': 1, 'b': 1, 'c': 5, 'd': 6, 'e': 1, 'f': 4, 'g': 2, 'h': 4, 'i': 1, 'j': 8, 'k': 6, 'l': 1, 'm': 4, 'n': 1, 'o': 1, 'p': 3, 'q': 11, 'r': 1, 's': 1, 't': 1, 'u': 1, 'v': 5, 'w': 4, 'x': 8, 'y': 6, 'z': 10, ' ':100

Please finish the “get_word_score” function for returning the correct word score (condition is given from function comments). The necessary library is given to you for importing (not other libraries should be used). Test with “Australia”, “Deakin” and your name for three cases.

```
def get_word_score(word):  
    """  
    Returns the score for a word. Assumes the word is a  
    valid word.  
  
    You may assume that the input word is always either  
    a string of letters ,  
    and can not be the empty string "".  
    You may not assume that the string will only contain  
    lowercase letters , so you will have to handle uppercase  
    and mixed case strings  
    appropriately.  
  
    The score for a word is the product of two components:  
  
        The first component is the sum of the points for letters  
        in the word.  
        The second component is the larger of:  
        1, or  
        7*wordlen - 3*(20-wordlen),  
        where wordlen is the length of the word  
  
    Letters are scored as in LETTER_VALUES.  
  
    word: string  
    returns: int >= 0  
    """
```

Question 1.8

Could you please use loop(s) with string character “*” to print as below Figure 3?

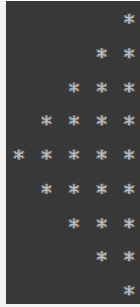


Figure 3: Program and Print the shape

Part II

Python Foundations for Data Science

This part is for students who are aiming to achieve ‘High Distinction’ (HD) for this assessment task. The Part II has been designed for a particular scenario based data science problems.

There are **2** versions in this part for **20** marks: **10** marks for coding, **5** marks for video presentation SIT742Task1Part2.avi (as in ‘**What to Submit**’) and **5** marks for the essay SIT742Task1Part2report.pdf (as in ‘**What to Submit**’).

For your question, you are required to use Google Colab to finish all the coding in the *code block cell*, provide sufficient coding comments, and also save the result of running.

1 Which question for you?

```
def sum_digits(n):
    r=0
    while n:
        r, n = r + n % 10, n // 10
    return r

def check_studentid(studentid):
    x = sum_digits(studentid)
    if x % 2 == 0:
        print ('version I')
    else :
        print ('version II')

print('Correct Version of Q2 for me is:')
#check_studentid ()
```

2 Question 2 - Version-I

Question 2: Calculate the Area between a curve and a line

There are two functions $y = 10 - 2x$ and $y = 5 + 4x - x^2$. The intersection of the two function is shown as below graph. It could be seen that the area (highlighted with red color) is where the $y = 5 + 4x - x^2$ value larger than $y = 10 - 2x$ like Figure 4. This scenario we could call it as “Area

between a curve and a line”. Now your task is to write few functions and draw some graphs to calculate this “Area between a curve and a line”.



Figure 4: The area between $y = 10 - 2x$ and $y = 5 + 4x - x^2$

Question 2.1

The first step is to find out the “the intersection of the curve and the line” like below graph:

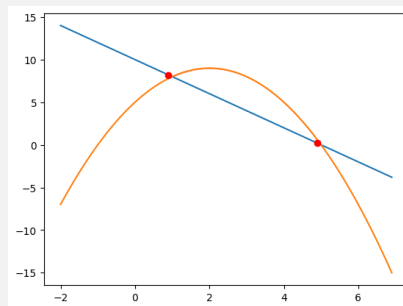


Figure 5: The intersection of $y = 10 - 2x$ and $y = 5 + 4x - x^2$

Could you please continue working in the graph function to draw “the intersections of the curve and the line” like Figure 5? Also could you print out the two intersections corresponding x and y pair value? (you could round the x and y pair value)

```
def graph(formula1, formula2, x_range):
    x = np.array(x_range)
    #use x as range variable
    y1 = formula1(x)
    y2 = formula2(x)
    #call the lambda expression with x on each
    #equation formula, it is given to you
    #use y1 and y2 as function result
    plt.plot(x, y1)
    plt.plot(x, y2)
    plt.fill_between(x, y2, y1, where=(y1 < y2),
                     color='red', alpha=0.3)
    # your code to fill below

    plt.ylim(ymin=0)
    plt.show()
    # your code to print the two intersections
    #x and y pair values in list or tuples [x1,y1]
    #and [x1,y1]?
```

```
#end of function , no return values
```

```
#run the graph function to test your code for marking  
graph(lambda x : (10 - 2*x), lambda x : (4*x - x**2) +5,  
np.arange(-2, 7, 0.0001))
```

Question 2.2

The second step is to find out “the area under the curve $y = 5 + 4x - x^2$ ” by using the two intersections like below Figure 6

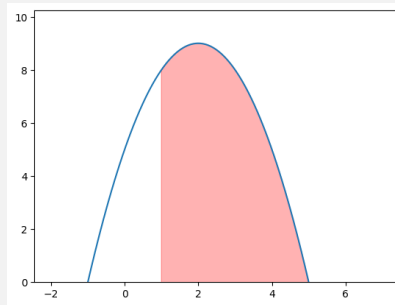


Figure 6: The the area under the curve $y = 5 + 4x - x^2$

You will need to modify the above **graph** function and draw the graph **exactly** like Figure 6. Also you need to finish the code and define the function **area_curve** and calculate the area under the curve $y = 5 + 4x - x^2$ with two intersections x and y pair values you obtained in **Question 2.1**.

Hint: to calculate “the area under the curve” you need to use *Trapezoidal Rule Formula*. You could find the steps to calculate in https://en.wikipedia.org/wiki/Trapezoidal_rule

```
def area_curve(p1,p2):  
    """  
    your code here , p1 and p2 are [x1,y1]  
    and [x2,y2] pair values  
    """
```

Question 2.3

The third step is to find out “the area under the line $y = 10 - 2x$ ” by using the two intersections like below Figure 7

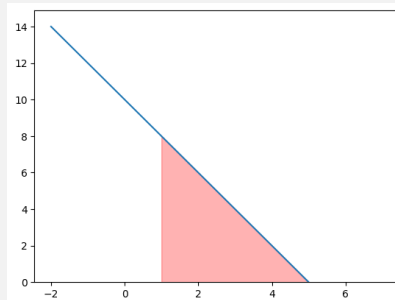


Figure 7: The the area under the line $y = 10 - 2x$

You will need to modify the above **graph** function and draw the graph **exactly** like Figure 7. Also you need to finish the code and define the function **area_line** and calculate the area under the line

$y = 10 - 2x$ with two intersections x and y pair values you obtained in **Question 2.1**.

```
def area_line(p1,p2):  
    """  
    your code here, p1 and p2 are [x1,y1]  
    and [x2,y2] pair values  
    """
```

Question 2.4

The last step is to subtract the two calculated area to obtain the “area between curve and line” Please run the above defined functions **area_curve** and **area_line** to return the value of the area between curve and line (you could round the result to 3 decimals).

3 Question 2 - Version-II

Question 2 Buying a House – how long you need to save for your initial payment?

You have moved to Melbourne to start your life and decide to get into the property market as soon as possible. The house price is high and you want to save the money for some years (months) before you could afford the down-payment (initial payment) of the house. In this part, we will program to calculate how long you need to save for make the payment of the house.

There are few rules (variables) you need to define to calculate:

- Call the cost of your home **total_cost**.
- Call the portion of the cost needed for a initial payment **portion_down_payment**. For simplicity, assume that **portion_down_payment** = 0.15(15%).
- Call the amount that you have saved so far as **current_savings**. You start with a current savings of 0.
- Assume that you invest your current savings wisely, with an annual return of r (in other words, at the end of each month, you receive an additional **current_savings** * $r/12$ funds to put into your savings – the r is an annual rate). Assume that your investments earn a return of $r = 0.04$.
- Assume your annual salary is **annual_salary**.
- Assume you are going to dedicate a certain amount of your salary each month to saving for the down payment. Call that **portion_saved**. This variable should be in decimal form.
- At the end of each month, your savings will be increased by the return on your investment, plus a percentage of your monthly salary (**annual_salary**/12).

Question 2.1

In this question, your **annual_salary** is fixed while you are saving for your house initial payment. To finish this question, you need to write a program in python (function or syntax code). Your program should ask user to enter the following variables and return the months you need to save:

- The starting annual salary (**annual_salary**).
- The portion of salary to be saved (**portion_saved**).
- The cost of your dream home (**total_cost**).

Question 2.2

We unrealistically assumed that your salary did not change. As you are Deakin Master student, clearly you will receive a raise on your salary with time! So we are going to build on your solution to **question 2.1** by factoring in a raise every 12 months.

In order to program it with a salary raise, you need to have additional input **annual_raise** (as a decimal percentage). We will assume your salary will raise every 12 months with your input percentage and calculate how many months it will take you save up enough money for a down payment. Like before, assume that your investments earn a return of $r = 0.04$ (or 4%) and the required down payment percentage is 0.15 (or 15%). Have the user enter the following variables and return the months you need to save

- The starting annual salary (**annual_salary**).
- The portion of salary to be saved (**portion_saved**).
- The cost of your dream home (**total_cost**).
- The annual salary raise (**annual_raise**).